Digital Workforce Twin

Project Overview

An agent-based organizational simulation platform for workforce retention decision-support: architecture, methodology, and current state.

A complete explanation of what this project is, how it evolved, how it's built, and what's in it today. Written to bring a new reader (or a future version of ourselves) fully up to speed without needing the conversation history that produced it.

1. What this is

An agent-based simulation platform that models a company's workforce as autonomous software agents, so an HR/people-ops decision-maker can test "what if we did X" - a layoff, a hiring wave, a reorg, a retention bonus - in a virtual company before doing it to a real one.

The project is anchored on one concrete decision-support question:

> ***Who is at risk of leaving, and which intervention most reduces that risk, with what confidence?***

Everything else - the synthetic data, the ML model, the Monte Carlo runner, the scenario library, the root-cause diagnosis, the webapp - is in service of that one question, not a grab-bag of unrelated demos.

2. How the scope got here

The original brief ("Digital Workforce Twin: agent-based org simulation + ML + Monte Carlo + MLOps") was broad enough to build almost anything, and early work did: an agent-based sim, synthetic data generation, ML models, a Monte Carlo runner, and a Streamlit dashboard, each built well but not obviously serving one deliverable.

The turning point was a direct question about whether the scope was too vague. The honest answer was yes, and the concrete symptom was in the ML layer: it trained a regressor to predict `turnover_risk` from `level`/`tenure`/`salary`/`team_size` - but those targets were a **closed-form function of a subset of those same features**, computed by the same generator. The model wasn't predicting behavior; it was inverting a formula that was already known. That's what "no objective" looks like in code.

The fix was to anchor everything on the retention-risk question above, which forced several honesty fixes along the way (see §5).

3. Architecture at a glance

```
Simulation engine (companysim.model, .agents, .scenarios, .data)
        |
        |---- used directly by --- Streamlit dashboard (companysim.dashboard)
        |---- used directly by --- CLI scripts (scripts/*.py)
        `---- used directly by --- FastAPI backend (companysim.api)
                                        |
                                        `---- React/TypeScript webapp (webapp/)
```

The simulation engine has never been rewritten for any of its three interfaces - the dashboard, the scripts, and the webapp are all thin adapters over the same `OrganizationModel`, `Scenario`/`Event` classes, and ML pipeline. This was a deliberate constraint: new interfaces should not require new simulation logic.

4. The simulation engine

4.1 Agents and organization structure

- **`data/schemas.py`** - the engine's internal contract: `Employee`, `Team`, `Department`, `Organization` (pydantic models, minimal fields).
- **`agents/employee.py`** - `EmployeeAgent` wraps an `Employee` record plus a `HumanProfile` (see below) and evolves both one tick (one simulated week) at a time in `step()`.
- **`agents/team.py`** - `TeamAgent` aggregates its members' engagement/ collaboration into a `climate` signal and their psychological-safety perceptions into a team-level score - both feed back into each member's next tick, which is what gives the sim its peer-effect character.
- **`model/organization.py`** - `OrganizationModel` is the top-level scheduler: applies any scenario events due that tick, recomputes team climates, then steps every employee in randomized order, and returns the tick's `SimulationSnapshot`.

4.2 The human-factors framework

`data/human_factors.py` generates each employee's psychological and life-context profile, explicitly grounded in named organizational- psychology instruments rather than invented numbers:

- **Big Five personality** - Costa & McCrae (1992)
- **Burnout** (exhaustion/cynicism/efficacy) - Maslach Burnout Inventory
- **Workload / autonomy / support** - Job Demand-Control-Support (Karasek)
- **Psychological safety** - Edmondson (1999)
- **Adverse Childhood Experiences score** - CDC-Kaiser ACE study
- **Attachment style** - Hazan & Shaver
- **Weekly pulse survey data** - modeled on CultureAmp/Peakon-style continuous listening, ~75% response rate

Two fields (ACE score, attachment style) are included purely as internal latent variables for population

realism - they are **not** things a real employer measures or should measure, and the module docstring says so explicitly.

`HumanProfile.from_row()` builds an agent's live wellbeing state directly from a `human_factors` table row, which is what makes the exported dataset and the running simulation describe the same population (see §5.1).

4.3 Per-tick dynamics

Each `EmployeeAgent.step()` evolves, in order: burnout (builds under workload, decays with sleep/manager support), stress, sleep quality, mood, engagement, collaboration, productivity, and turnover risk - each a persistence-weighted update (mean-reverting toward a baseline plus peer/team influence plus noise). Turnover risk feeds a convex hazard function for the actual weekly quit draw - mild dissatisfaction rarely triggers an exit, but risk crossing a threshold does, matching the empirical "tipping point" pattern in real voluntary-turnover data.

---

5. Honesty fixes made along the way

These are worth documenting because they were real bugs/design flaws caught and fixed during the reframe, not hypothetical concerns:

5.1 Two populations that silently diverged

The dataset export (`data/human_factors.py`'s `generate_human_factors`) and the simulation's per-agent state (`HumanProfile.sample()`) were independently sampled from similar-but-different formulas. Fixed by `HumanProfile.from_row()` + a `DatasetBuilder -> Organization` bridge (`data/datasets.py::to_organization`), so "features from the dataset" and "labels from forward-simulating that dataset" describe the same people.

5.2 A real quit-counting bug

`quits_this_tick` counted every already-departed employee again on every subsequent tick, because `EmployeeAgent.step()` returns `quit=True` forever once inactive. This made cumulative-quit numbers wildly wrong (764/2000 instead of ~135/2000) until fixed by tracking the active->inactive *transition* instead of trusting the per-tick flag.

5.3 Weak predictive signal, diagnosed and calibrated

The original quit-probability curve and state persistence washed out day-0 signal almost entirely (AUC ~ 0.55, indistinguishable from noise). Diagnosed by comparing an "oracle" ceiling (AUC using the raw internal risk latent directly) against the achievable model AUC; fixed by raising state persistence (burnout, turnover risk) and steepening the quit-hazard curve (cubic -> power 4). Final realistic AUC ~ 0.6-0.7 - high enough to be useful, nowhere near the ~0.99 that would signal leakage.

5.4 Company-wide totals are the wrong denominator for a targeted program

If an intervention touches 120 of 2,000 employees, the other 1,880 employees' quit noise swamps any signal from the 120. `intervene.py`'s `compare_intervention` tracks outcomes *within the targeted cohort* specifically - both the statistically correct choice and the one a real retention program gets judged on.

5.5 A locale formatting bug in the webapp

`toLocaleString()` without an explicit locale rendered a dollar cost as `$185 236,415` instead of `$185,236` depending on the browser's default locale. Fixed by passing `"en-US"` explicitly.

5.6 Conflating layoffs with voluntary quits

`Layoff` and `Termination` scenario events deactivate an employee directly (`agent.active = False`), completely different from the organic, Bernoulli-driven quit in `EmployeeAgent.step()` - but nothing outside `OrganizationModel` could originally tell the two apart. That meant a scenario with an injected layoff could generate a fabricated "I was burned out" exit note (§9.1) for someone who was actually just let go, and would have silently poisoned the MLOps training data (§11) by teaching the model that being laid off looks like voluntary attrition. Fixed by adding `OrganizationModel.organic_quit_ids`, which reuses the model's existing (but previously unexposed) active->inactive *transition* tracking - events fire before that tracking runs each tick, so an event-deactivated employee never registers as an organic quit.

6. The ML layer

6.1 Feature engineering - the leakage boundary

`ml/turnover_features.py` builds features from things a real HRIS or pulse-survey platform could actually observe *before* the outcome window: job/comp facts (level, department, role, tenure, salary, team size, manager status, promotions) plus rolling aggregates of pulse-survey data and performance ratings. It explicitly excludes the simulation's internal latents (`engagement`, `collaboration`, `productivity`, `turnover_risk`) - including them would reintroduce the exact circularity the reframe was about removing. `assert_no_leakage()` is a standing guard against regression.

6.2 Label generation - real simulated events

`ml/turnover_labels.py` generates labels by forward-simulating a population for a horizon (default 12 weeks) and recording who *actually quit* (a real stochastic event from `EmployeeAgent.step()`), not a hand-set score. Multiple replicates per population give the same day-0 features several independent stochastic label draws, which is what keeps a trained model from learning a deterministic mapping.

6.3 Training + registry

`ml/train.py` trains a `GradientBoostingClassifier` in a `ColumnTransformer` (one-hot + scaling) pipeline; **`ml/registry.py`** persists it as a `TurnoverModelBundle` via joblib. A secondary `BehavioralModelBundle` (productivity/engagement/collaboration regressors) is kept as an auxiliary trajectory-preview tool - its module docstring is explicit that it is *not* the evaluated deliverable, since its targets are a closed-form function of a subset of its own features.

6.4 MLOps promotion gate

`ml/gate.py` (shared by `scripts/train_turnover_model.py` and the webapp's `/model/train` endpoint):

1. Build a fresh training cohort from a rolling seed.
2. Train a candidate model.
3. Evaluate it on a **fixed** held-out cohort (seed never changes - a stable benchmark across retrains).
4. Load the current production bundle (if any) and evaluate it on the same fixed cohort.
5. Promote the candidate only if it doesn't regress AUC beyond a tolerance (default 0.02); otherwise leave production untouched.

Every run appends an audit line to `models/turnover_promotion_log.jsonl`.

---

7. Monte Carlo + the scenario library

`mc/runner.py`'s `MonteCarloRunner` runs a scenario N times (each replicate deep-copies the starting population, varies only the stochastic seed) and aggregates into p05/p50/p95 percentile bands per metric per tick - what a policymaker actually needs to read a scenario: not the mean outcome, but the range of plausible ones.

`scenarios/events.py` - 12 events, one `Event` protocol (`at_tick` + `apply(model)`), three categories:

- **Global / org-wide** - `Layoff`, `Hire`, `Promotion`, `PolicyChange`, `BudgetCut`, `Reorg`
- **Employee-targeted** - `RetentionBonus`, `WorkloadRelief`, `ManagerCoaching`, `Termination`, `Transfer`
- **Outside-of-work** - `LifeEvent` (bereavement, birth/adoption, moving, divorce, illness, caregiving onset, financial shock)

`LifeEvent` is the direct payoff of the human-factors framework: it makes "things happen outside work that affect performance" something a scenario can actually trigger, not just a static generation-time attribute.

---

8. The what-if / intervention workflow

`intervene.py`:

- `rank_at_risk(bundle, features)` - sorts employees by predicted turnover probability.
- `compare_intervention(...)` - runs paired baseline/treated Monte Carlo replicates (same population, same per-replicate seed, diverging only at the intervention's tick - a lower-variance comparison than independent sampling), measuring quits avoided *within the targeted cohort* (§5.4) plus a cost estimate.

---

9. Root-cause diagnosis

`ml/diagnostics.py` - detects a problem in a run, attributes a root cause, and recommends a fix. Four independently-testable stages:

1. **`detect_problems`** - threshold-crossing checks (burnout rate, engagement drop, turnover-risk rise, quit spike) over a run's history. Full detail, including exact formulas and calibration rationale, in `docs/diagnosis_thresholds.md`.
2. **`diagnose`** - ranks (department/team, feature) pairs by importance-weighted deviation from the org-wide mean, reusing the trained turnover model's own `feature_importances_` as weights (falls back to equal weighting if no model is available). A transparent, dependency-free substitute for SHAP - every claim is a checkable mean and a subtraction, not a black-box score. Deliberately adapter-agnostic: works on both the webapp's DB-backed fields and the offline pipeline's pulse-trend features.
3. **`recommend`** - a rule-based lookup from the top driving feature to one of the 12 scenario events, targeted at the worst-affected employees in the diagnosed segment.
4. **`explain`** - template-based natural language generation over the structured findings. Deliberately not an LLM call: there's no free-text corpus in this synthetic system to justify one, and generating a sentence from known facts is the right-sized tool.

Known, stated simplification: root-cause attribution uses the org's *current* driver state, not a replay of per-employee state at the exact problem tick (the engine only snapshots aggregate metrics per tick, not full per-employee history) - the same limitation a real people-analytics team has reading a current HRIS/survey snapshot instead of a full replay.

9.1 Real NLP over synthetic exit notes

`ml/exit_notes.py` upgrades `explain()`'s template-only output with a genuine text layer, split into two deliberately separate halves:

- **Generation** - `generate_note()` builds a short first-person exit note grounded in an employee's actual driver values, reusing `BAD_DIRECTION` (the same "which direction is bad" convention `diagnose()` uses) so the note's content is honestly tied to the same underlying state, not independently

invented.

- **Extraction** - `analyze_notes()` runs real text analysis over a set of notes: TF-IDF keyword extraction (scikit-learn), a small hand-built sentiment lexicon, and a keyword->theme mapping *independent* of the structured diagnosis, so agreement between the two is genuine corroboration rather than the same computation twice. Classical, dependency-light NLP by design - no free-text corpus exists in this synthetic system to justify a transformer or LLM call.

Wiring: after a diagnosis run, employees who *organically* quit (see §5.6) within the diagnosed segment get notes generated and analyzed; the result augments the explanation with a real qualitative paragraph and sample quotes. If nobody in the segment actually left during that run, the explanation says so honestly ("based on structural risk factors alone") instead of fabricating sentiment.

9.2 Exportable PDF reports

`POST /orgs/{id}/diagnose/export` (`api/pdf_report.py`, built on `fpdf2` - pure Python, no system-level dependencies) renders a full diagnosis report - per-problem description, drivers, recommendation, and the notes-augmented explanation with sample quotes - as a downloadable PDF. Shares `build_diagnosis_report()` with the JSON `/diagnose` endpoint (same pattern as `scenario_builder.py`), so the two never drift apart.

---

10. The webapp

React + TypeScript (Vite) frontend, FastAPI backend, SQLite persistence - built to replace Streamlit's "rerun everything, nothing persists" model with a real edit/run/observe loop.

10.1 Backend (`src/companysim/api/`)

- **`database.py`, `db_models.py`** - SQLAlchemy engine + ORM (`OrgRecord`, `DepartmentRecord`, `TeamRecord`, `EmployeeRecord`, `EmployeeWellbeingRecord`, `RunRecord`, `TurnoverTrainingExample`)
- **`seed.py`** - creates a new org by running `DatasetBuilder` once and persisting every row
- **`converters.py`** - DB rows to the pydantic `Organization` + human-factors DataFrame the engine consumes (mirrors `data/datasets.py::to_organization`)
- **`scenario_builder.py`** - wire-format event list to `Scenario` objects (shared by `/simulate` and `/diagnose`)
- **`scoring_frame.py`** - DB org to a `ml.turnover_features.FEATURE_COLUMNS`-shaped frame (shared by `at_risk.py`'s live scoring and `training_examples.py`'s example collection)
- **`run_history.py`** - persists every simulate/diagnose call (`RunRecord`) so it can be reopened later
- **`training_examples.py`** - collects labeled turnover examples from real runs; reshapes them for `ml/gate.py` (see §11)
- **`pdf_report.py`** - renders a diagnosis report to PDF (`fpdf2`)
- **`routers/orgs.py`, `departments.py`, `teams.py`, `employees.py`** - standard CRUD
- **`routers/simulate.py`** - single run or Monte Carlo, depending on `replicates`; collects a training

example per employee on qualifying single runs
- **`routers/diagnose.py`** - runs a simulation, then the diagnostics + exit-notes pipeline; `/diagnose/export` streams the PDF version
- **`routers/at_risk.py`** - ranks employees via the trained model (with a documented pulse-approximation adapter for the schema gap - see below) + intervention comparison
- **`routers/model.py`** - wraps `ml/gate.py`, not org-scoped since the model is meant to generalize, but blends in every org's collected training examples
- **`routers/runs.py`** - browse/reopen/delete saved run history

The at-risk scoring schema gap, and how it's handled: the trained turnover model expects pulse-survey trend features and performance ratings the webapp's DB doesn't persist. Rather than silently faking richer history, `scoring_frame.py::build_scoring_frame` uses the employee's *current* wellbeing snapshot as a stand-in for a trailing average, trends default to 0, ratings default to a neutral 3.0, and `department_id` (the one field that's genuinely lossy - the model learned the offline pipeline's string ids, not the webapp's integers) simply contributes nothing to that one-hot slice rather than erroring. This is documented in the module's docstring, not hidden - and the same adapter is what makes real webapp usage collectible as training data (§11).

10.2 Frontend (`webapp/src/`)

- **`OrgListPage`** (`/`) - create/list orgs
- **`OrgEditorPage`** (`/orgs/:id`) - CRUD employees/departments/teams
- **`SimulatePage`** (`/orgs/:id/simulate`) - scenario builder (all 12 events) + fan charts + Diagnose + PDF export
- **`AtRiskPage`** (`/orgs/:id/at-risk`) - ranked risk table + intervention comparison
- **`RunHistoryPage`** (`/orgs/:id/runs`) - browse and reopen past simulate/diagnose runs
- **`TrainModelPage`** (`/model`) - production model status, pending live-example count, retrain

`FanChart.tsx` is the one shared visualization - a percentile-band chart (Recharts `Area` + `Line`) reused for both single-run and Monte Carlo results. `SimulationResults.tsx`/`DiagnosisResults.tsx` are shared result-rendering components used by both `SimulatePage` (a live run) and `RunHistoryPage` (a reopened one), so both render identically.

10.3 The Streamlit dashboard

Kept alongside the webapp (`companysim/dashboard/app.py`), not replaced - still useful for quick Monte Carlo sweeps over company-wide scenarios.

---

11. MLOps: learning from live usage

The model doesn't just sit still between manual retrains - every real simulate/diagnose run in the webapp contributes a genuine labeled example, and every retrain blends all of them in before deciding whether to promote.

Collection (`api/training_examples.py::collect_training_examples`): after `routers/simulate.py` (single, non-Monte-Carlo runs only - a replicated run has no single representative outcome) or `routers/diagnose.py` runs a simulation with `ticks >= 6` (short horizons don't give the quit mechanic enough time to mean anything), it builds the same `scoring_frame.build_scoring_frame` feature snapshot used for live at-risk scoring, and for each employee records whether they *organically* quit (`OrganizationModel.organic_quit_ids`, §5.6) - never conflating an injected Layoff/Termination with a real voluntary exit - into a new `TurnoverTrainingExample` row.

Blending (`ml/gate.py::run_training_gate`'s new `extra_examples` parameter): stays fully DB-agnostic - it only ever receives an already-built DataFrame, never a database session, preserving the module's existing separation from the webapp layer. The webapp's `/model/train` endpoint loads every collected example (`api/training_examples.py::load_collected_examples`, reconstructing the constant placeholder columns - trend/rating - that aren't worth storing per-row) and passes it in; the CLI (`scripts/train_turnover_model.py`) has no DB and simply omits it, training exactly as before.

Safety net, unchanged: the blended candidate still only gets promoted if it doesn't regress AUC beyond tolerance against the same fixed holdout - real usage data can only ever help or be a no-op, never silently degrade production. `/model/status` and the Train Model page surface how many live examples are pending and how many were blended into the last retrain, so the causal link from "using the app" to "the model learning" is visible, not hidden in a background job.

Deliberately **not** built: fully automatic background retraining (a scheduling/locking cost not worth it against a UI a person already visits to retrain manually), and any retention cap on collected examples (fine at this project's scale).

12. Testing

102 automated tests across 16 files (`tests/`), covering: the generator's determinism, the org-graph's integrity (no cycles, every FK resolves), directional correctness of every scenario event, the diagnostics pipeline's detection/ranking/recommendation logic, the exit-notes NLP layer's grounding and sentiment correctness, the turnover model's leakage guard and realistic-AUC band, the MLOps collection/blending pipeline (including that an injected Termination never gets mislabeled as an organic quit), and the full FastAPI surface (CRUD round-trips, simulate/diagnose/export/at-risk/train/run-history endpoints) via `TestClient` against an isolated in-memory database.

Every feature was also verified live in an actual browser (not just unit tests) - creating orgs, editing fields and confirming persistence via direct API checks, running scenarios and reading the resulting charts, downloading and reopening PDF/run-history artifacts, and running full training jobs (including with live examples blended in) and reading the promotion decision.

13. Tech stack

- **Python 3.13**, **Pydantic**, **NumPy/Pandas** - simulation engine and schemas
- **FastAPI** + **SQLAlchemy 2.0** + **SQLite** - webapp backend
- **React 19 + TypeScript** (Vite), **TanStack Query**, **React Router**, **Recharts** - webapp frontend
- **scikit-learn** (`GradientBoostingClassifier`), **joblib**, optional **MLflow** - ML
- **fpdf2** - PDF report generation
- **Streamlit + Plotly** - secondary dashboard
- **pytest** + FastAPI `TestClient` (via `httpx2`) - testing
- **Faker** - synthetic identity/demographic data

14. What's next

Ideas discussed but not yet built:

1. Per-employee root-cause explanations, not just per-department/team